PROTOCOLS: ELLIPTIC CURVE DIFFIE-HELLMAN, WPA2, WPA3 AND THE SIGNAL PROTOCOL

Tom Chothia

Introduction

- Elliptic Curve Diffie Hellman
  - This is what we really use, very few systems use the classic version.
- WPA2: the current standard wifi security protocols
  - Password attacks.
- WPA3: the future wifi security standard.
  - An improvement on WPA2
- The Signal Protocol.

Diffie-Hellman

- Diffie-Hellman is a widely used key agreement protocol.
- It relies on some number theory:
  - a mod b = n where for some “m” : a = m.b + n
- The protocol uses two public parameters
  - generator “g” (often 160 bits long)
  - prime “p” (often 1024 bits long)

Diffie-Hellman

- Alice and Bob pick random numbers r_A and r_B and find "$t_A = g^{r_A} \bmod p$" and "$t_B = g^{r_B} \bmod p$"

- The protocol just exchanges these numbers:
  1. $A \rightarrow B : t_A$
  2. $B \rightarrow A : t_B$

- "Alice" calculates "$t_B{}^{r_A} \bmod p$" and "Bob" "$t_A{}^{r_B} \bmod p$" this is the key:
  - $K = g^{r_A r_B} \bmod p$

Diffie-Hellman

r^A times

1. $A \rightarrow B : (\ g\ x\ g\ x\ ..\ x\ g\ )\ \text{mod}\ p$

r^B times

2. $B \rightarrow A : (\ g\ x\ g\ x\ ....\ x\ g\ )\ \text{mod}\ p$

Alice calculates

r^A times

r^B times r^B times r^B times

$(g\ x \ldots x\ g)\ \text{mod}\ p\ x\ (g\ x \ldots x\ g)\ \text{mod}\ p\ x\ \ldots\ x (g\ x \ldots x\ g)\ \text{mod}\ p$

$= (g\ x\ g\ x\ g\ \ldots x\ g)\ \text{mod}\ p$

$r^B r^A$ times

Diffie-Hellman

r^A times

1. $A \rightarrow B : (\ g \times g \times .. \times g\ ) \bmod p$

r^B times

2. $B \rightarrow A : (\ g \times g \times .... \times g\ ) \bmod p$

Bob calculates

r^B times

r^A times r^A times r^A times

$(g \times ... \times g) \bmod p \times (g \times ... \times g) \bmod p \times \ldots \times (g \times ... \times g) \bmod p$

$= (g \times g \times g \ldots \times g) \bmod p$

$r^A r^B$ times

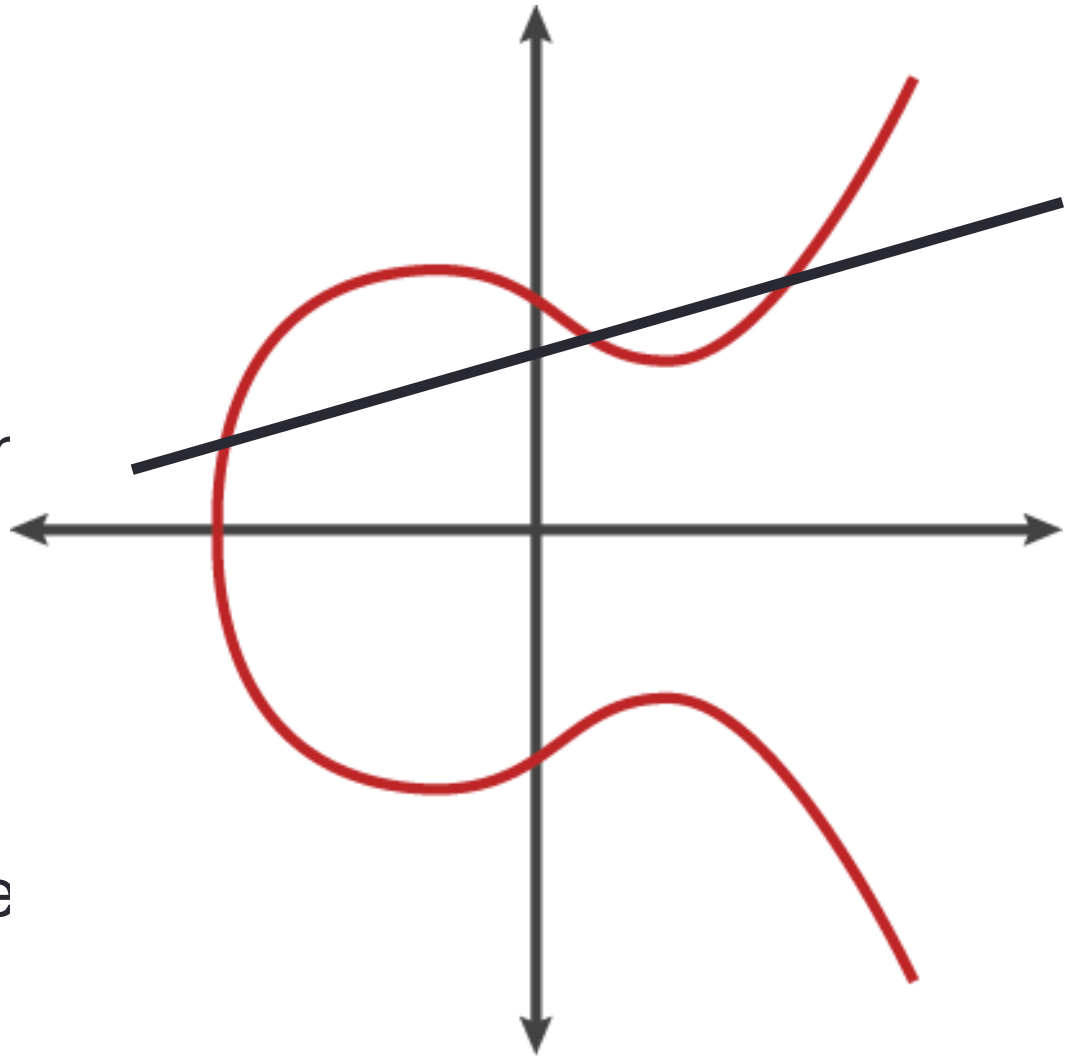
Elliptic Curves

- Elliptic curves are defined by an equation:

$$y^2 = x^3 + ax + b$$

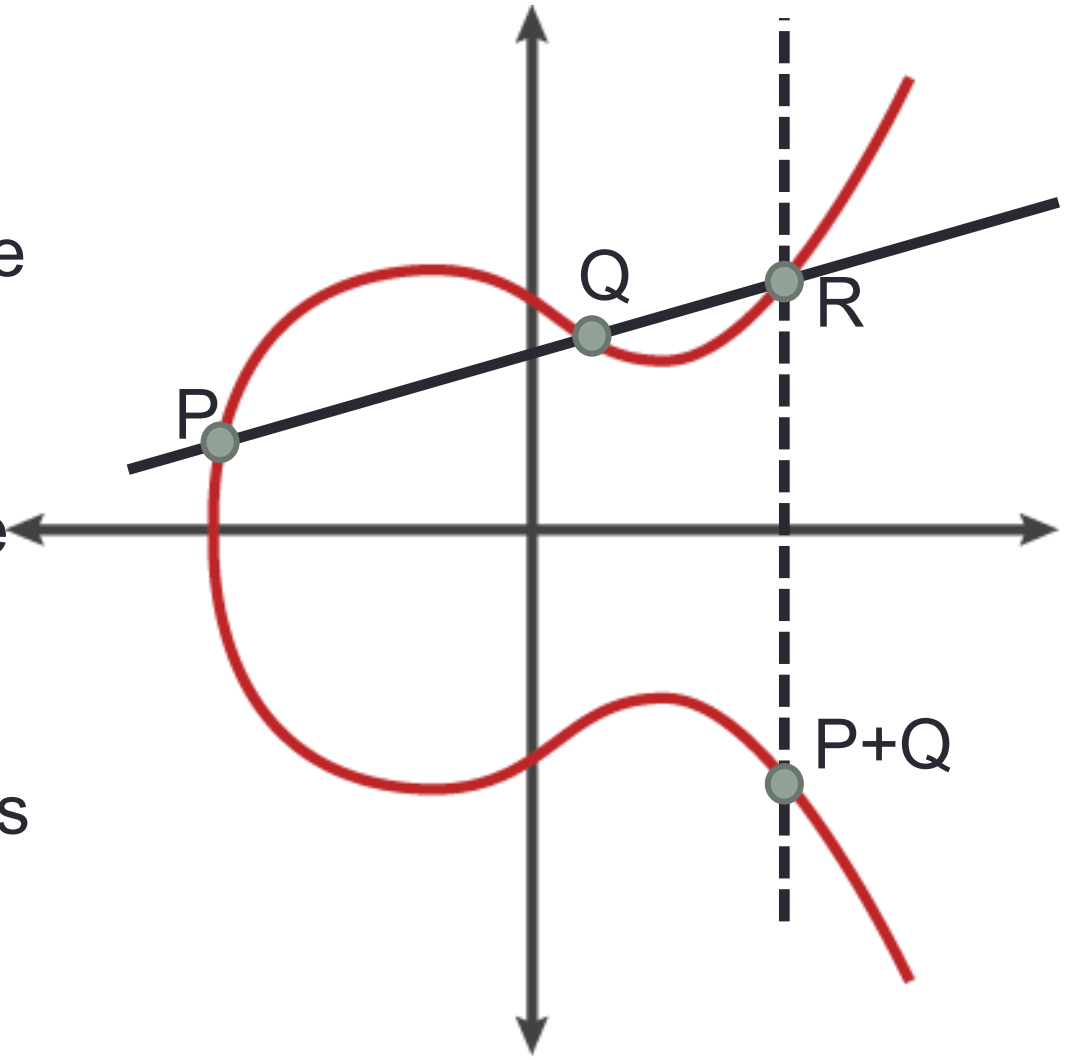
- A line through two point or the curve will cross one other.

- We can use this to define a + operation on the curve



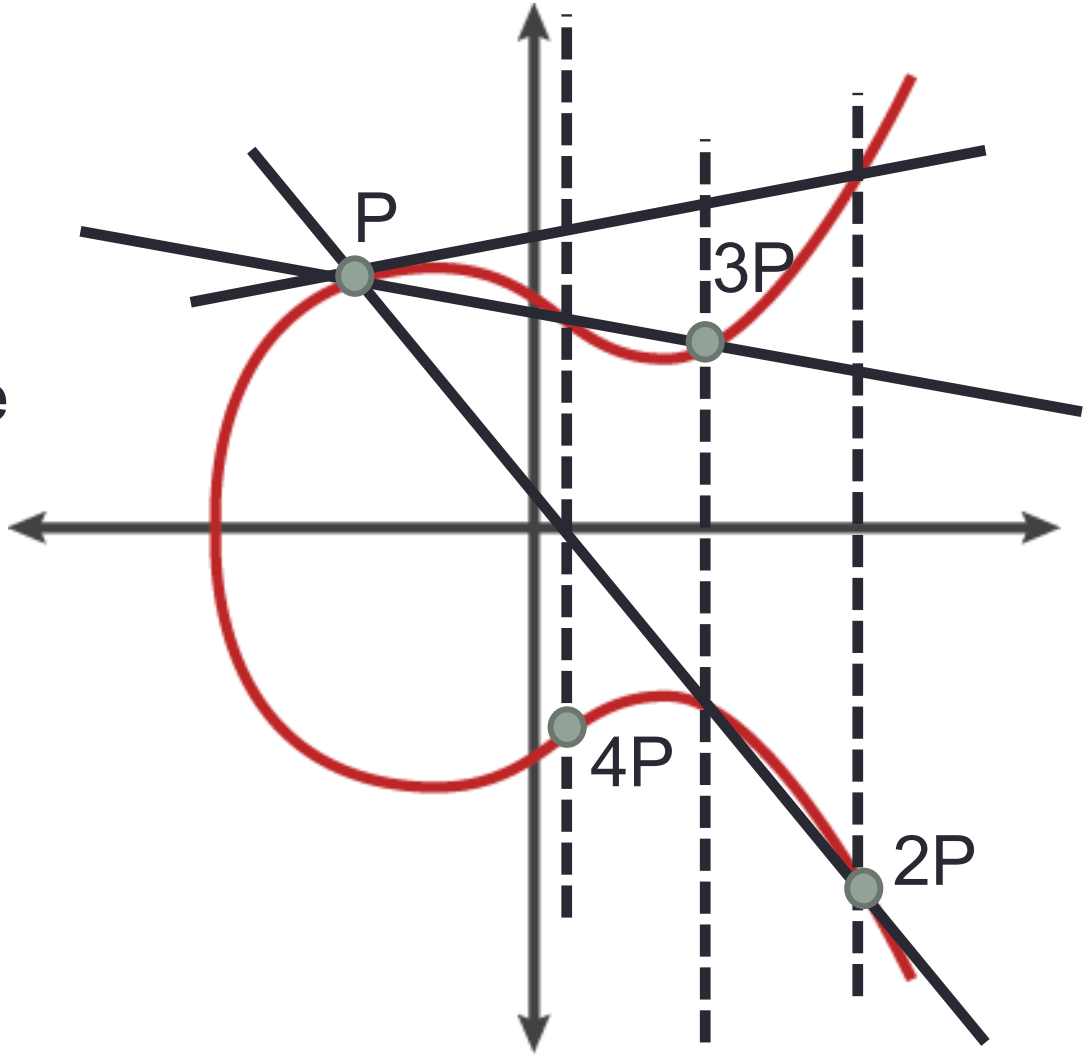
Elliptic Curves: the + Operation

- Given 2 points on the curve P and Q
- We find P + Q by draw line through P & Q
- The other point where line cross the curve is R
- P+Q = R reflected in x axis



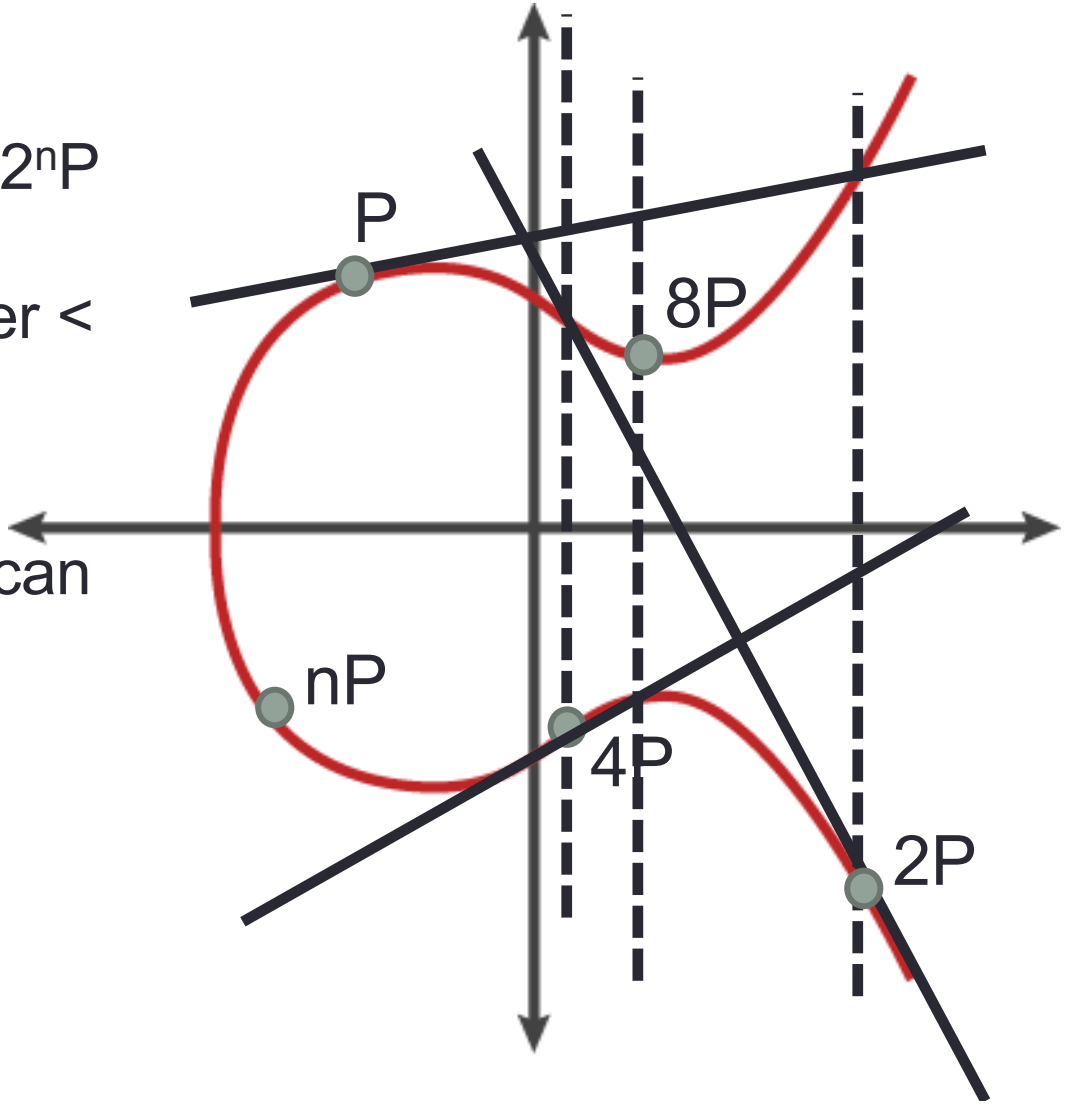
Elliptic Curves: the + Operation

- To find P + P we take the tangent
- We reflect this to get 2P
- We can then draw the line again to calculate 3P…
- This is a well defined +
  - p1+p2=p2+p1,
  - p1+(p2+p3) = (p1+p2)+p3
  - nP+mP = (n+m)P



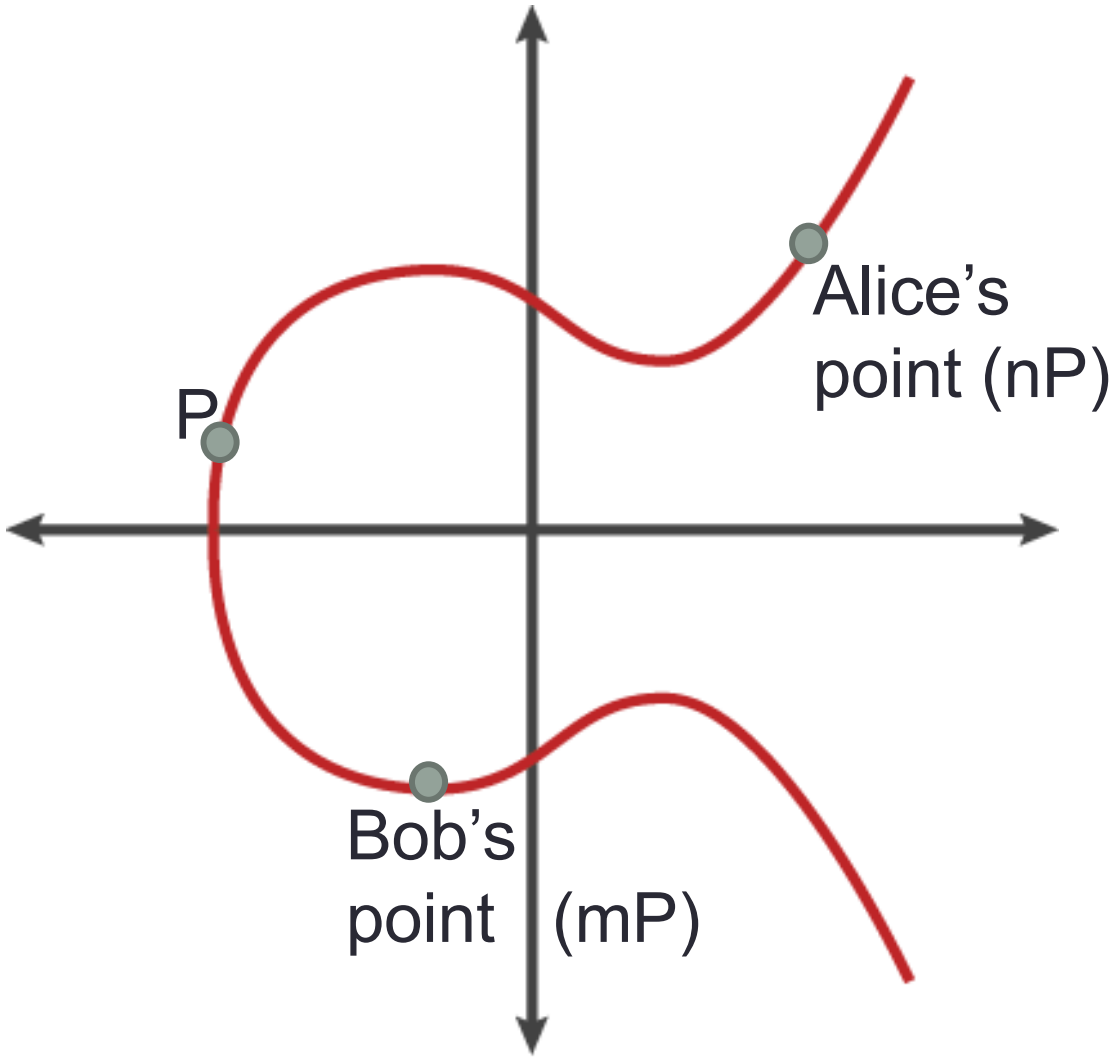
Elliptic Curve Operation: Doubling

- Doubling values is easy.
- We can calculate 2P, 4P, … 2^nP in n steps. Then add them together to make any number < $2^{n+1}P$
- If n is a 256-bit number, we can find nP ~ 500 operations.
- **If we don't know n finding it from nP takes 2^{256} operations.**



Elliptic Curve Diffie Hellman (ECDH)

- Alice and Bob agree on a curve.
- They agree on a point P.
  - P & curve are public.
- Alice picks random n and sends nP to bob
- Bob picks random m and sends mP to Bob



Elliptic Curve Diffie Hellman (ECDH)

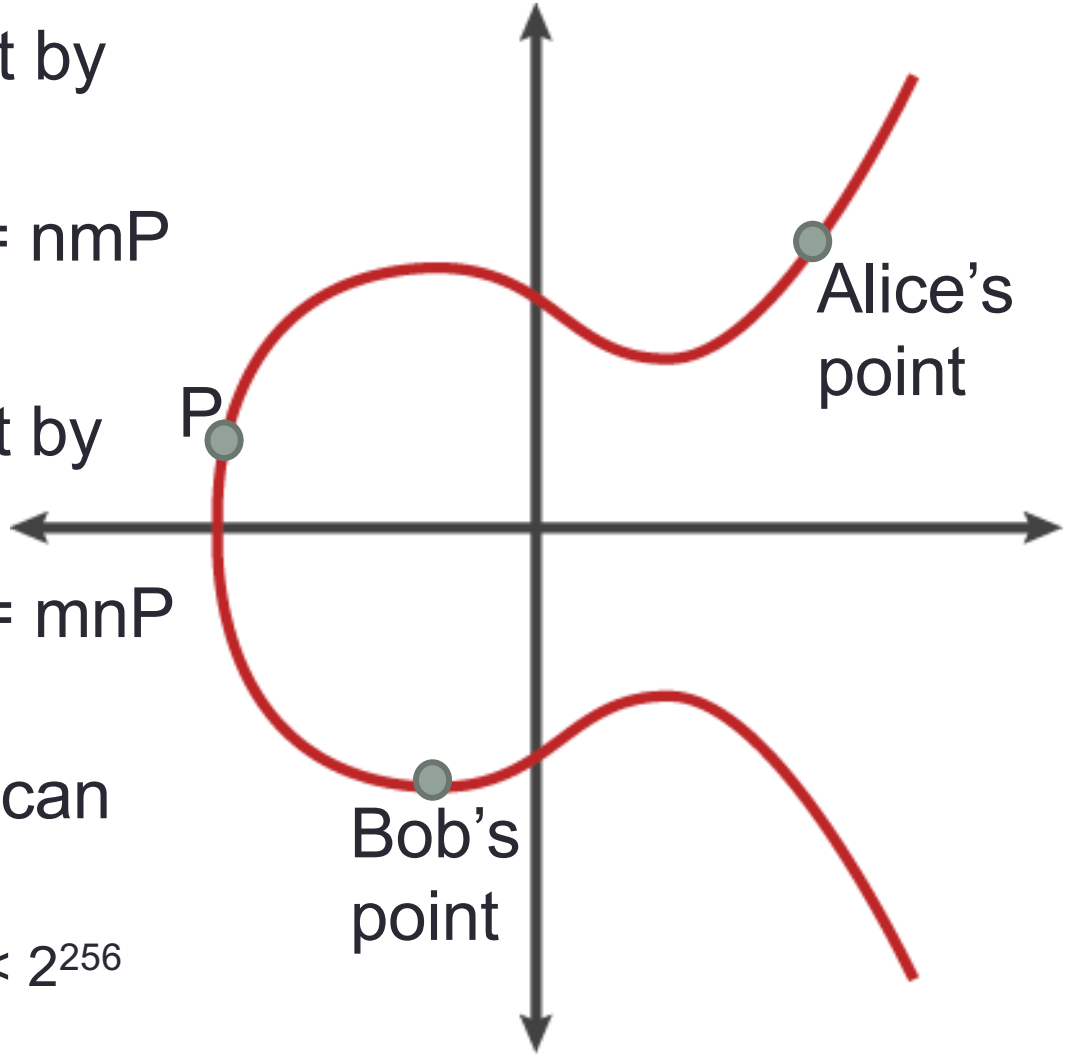
- Alice multiples Bob's point by her secret number:

  n (bob's point) = n(mP) = nmP

- Bob multiples Alice's point by her secret number:

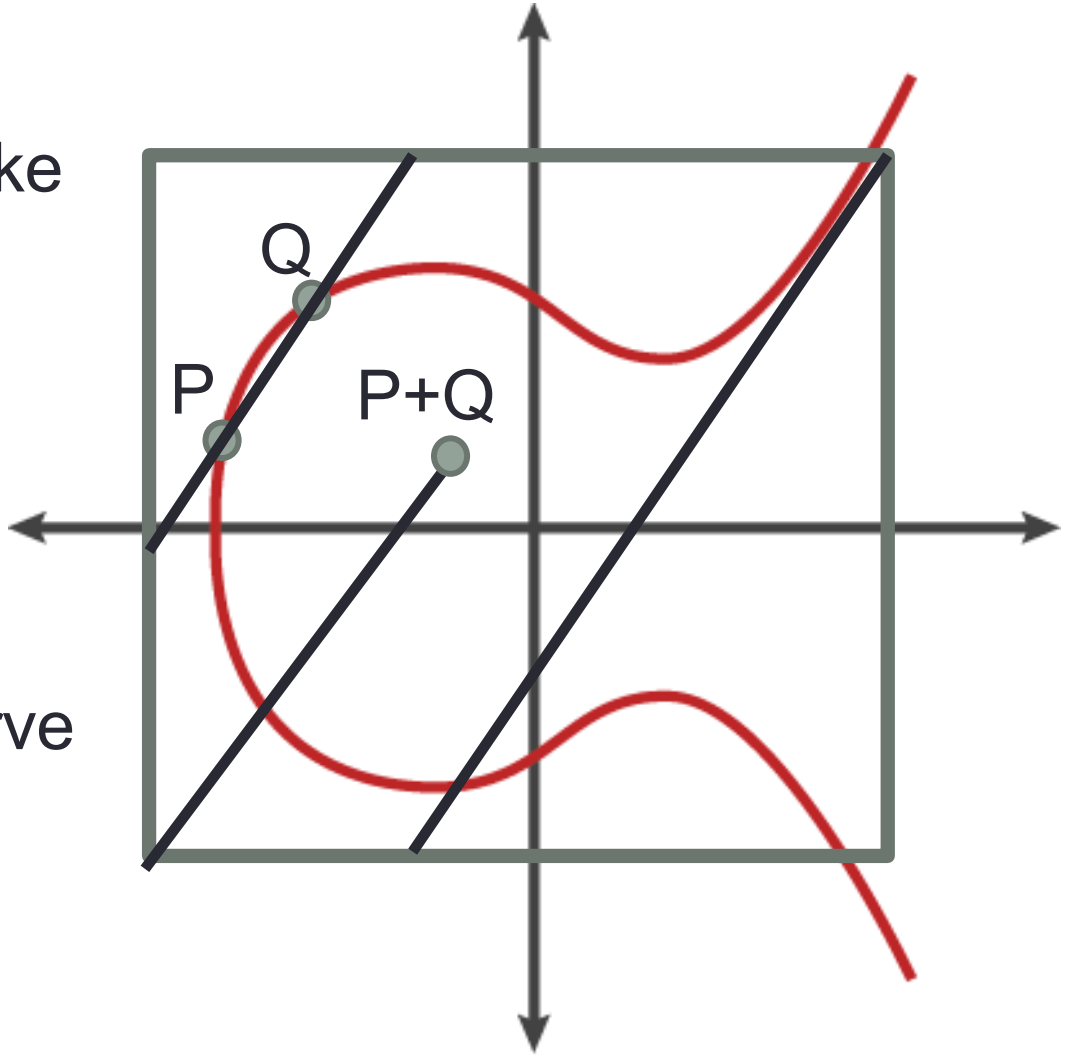
  m(alices's point) = m(nP) = mnP

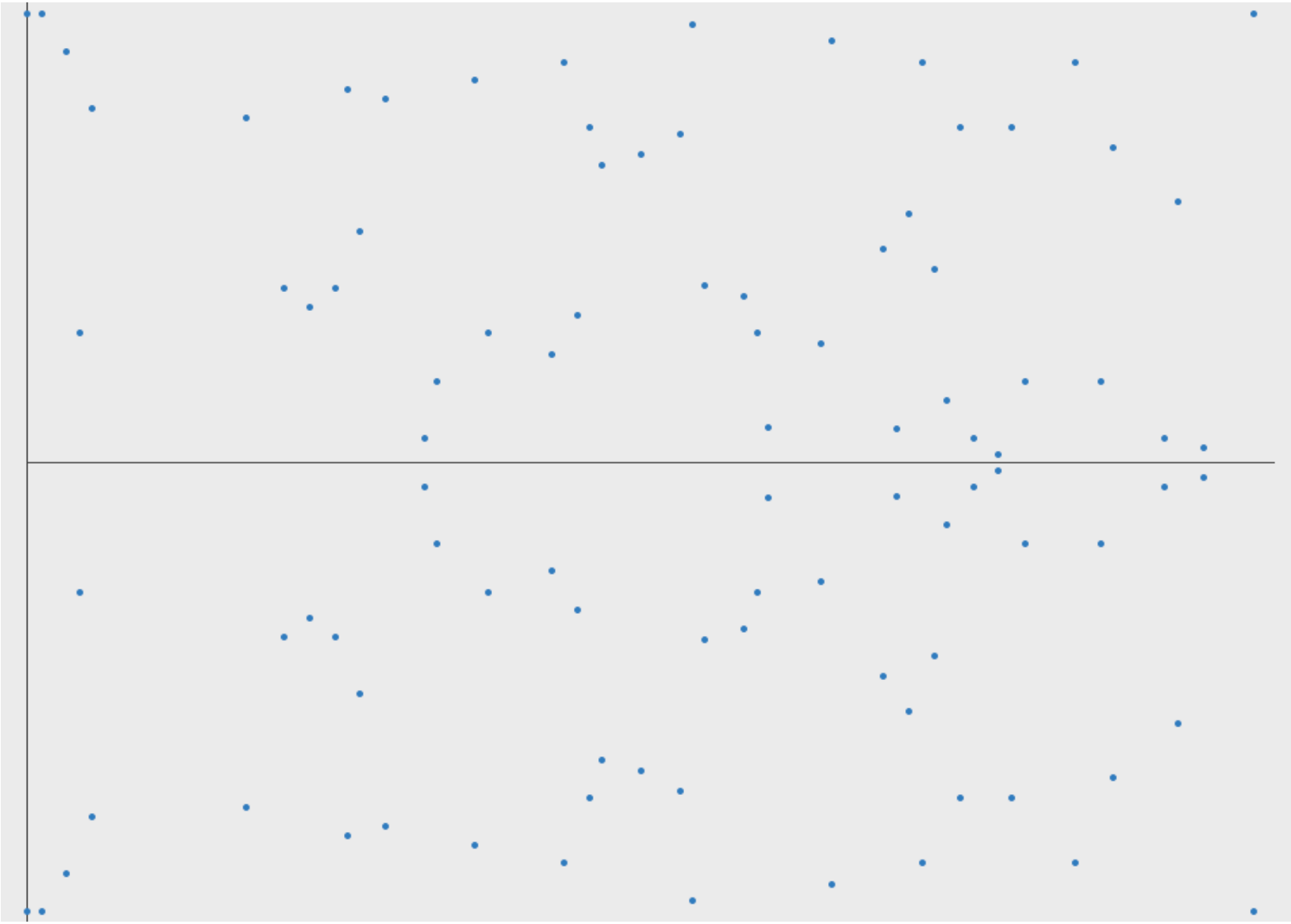
- The only way an attacker can find n or m is brute force.
  - Typically these are random < 2^{256}



Elliptic Curve Diffie Hellman (ECDH)

- Points might get very big!
- Just like classic DH we take values mod p.
- This limits the number of valid points.
- And we loose the nice curve shape.





https://arstechnica.com/information-technology/2013/10/a-relatively-easy-to-understand-primer-on-elliptic-curve-cryptography/

Picking a Curve:

- We must have a good curve, and valid point on curve.
  - I.e., no small subgroups, if 16P = P we get no security.
  - We must check that the values receives are valid curve points.
- **“Curve 25519”** is believed to be good one: https://en.wikipedia.org/wiki/Curve25519
- **“Brainpool”** and “**NIST**” curves are also well respected.
- **“Dual_EC_DRBG”** proposed by the NSA as a standard. Had subtle flaws that meant the NSA could crack it.

Diffie-Hellman DH vs ECDH

- Classic: generator “g”, prime “p” (often 1024 bits long)
  1. $A \rightarrow B : g^{r_A} \bmod p$
  2. $B \rightarrow A : g^{r_B} \bmod p$

Alice calculates $(g^{r_B} \bmod p)^{r_A}$ mo
Bob calculates $(g^{r_A} \bmod p)^{r_B}$ mo

Long Keys
Slow

- Elliptic curve parameters q,b,q, and point P (often 256 bits)
  1. $A \rightarrow B : r_A P$
  2. $B \rightarrow A : r_B P$

Alice calculates $r_A(r_B P) = (r_A r_B)P$
Bob calculates $r_B(r_A P) = (r_B r_A)P = (r_A r_B)P$

Short Keys
Fast

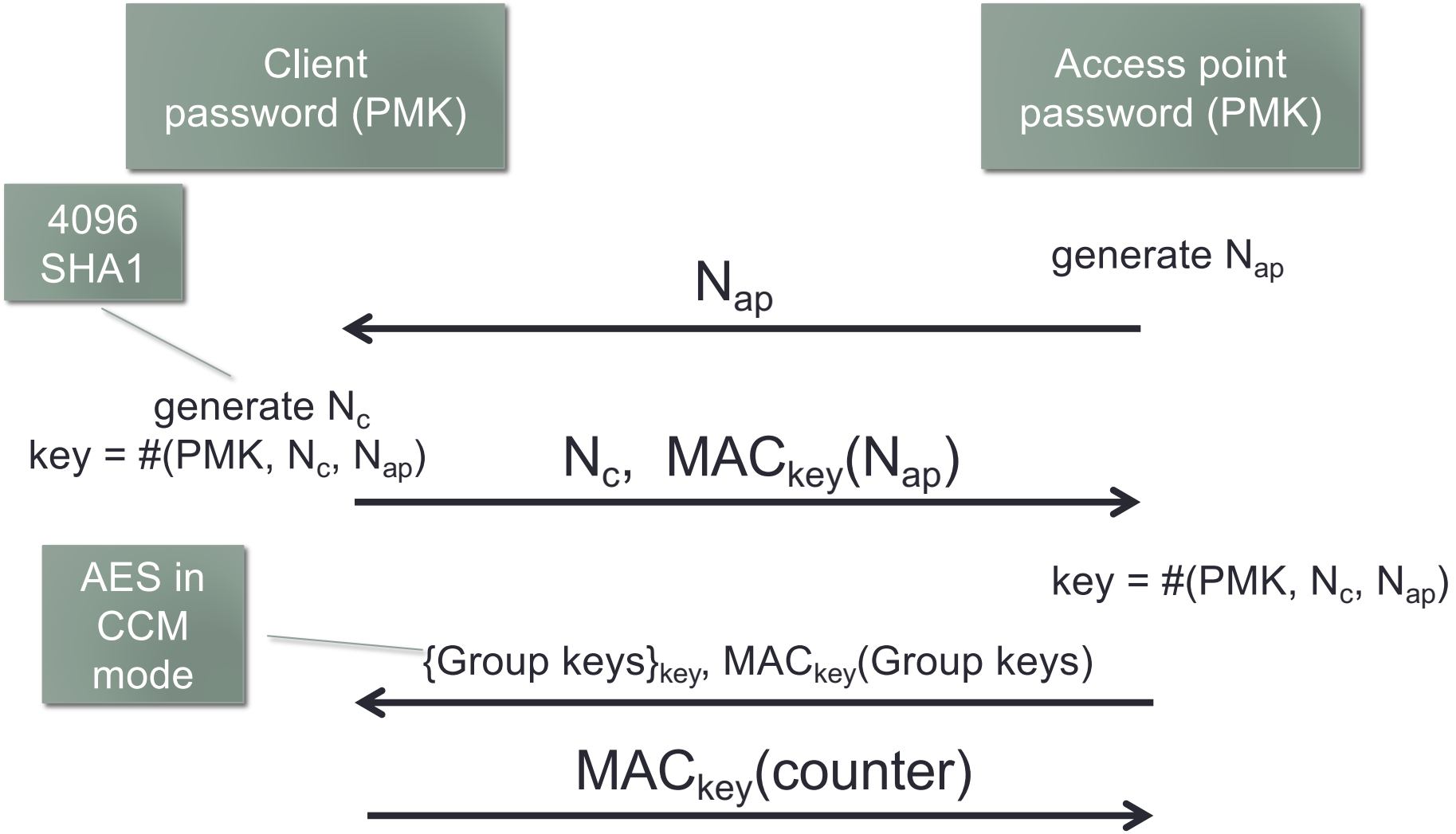
More Sources

- These slides cover everything you need to know.
- Lots of good sources online: youtube videos, blogs wikis
  - However, I have not found any that covers everything.
  - A lot live out the double operation!
- Free textbooks:
  - **Understanding Cryptography**, Chapter 9, http://swarm.cs.pub.ro/~mbarbulescu/cripto/Understanding%20Cryptography%20by%20Christof%20Paar%20.pdf
  - **A Graduate Course in Applied Cryptography**, Dan Boneh and Victor Shoup, Chapter 15, https://crypto.stanford.edu/~dabo/cryptobook/BonehShoup_0_5.pdf
- However, these have much more maths detail than needed.

WPA2 wi-fi security

- Wi-fi security protocols developed by a closed, paid membership group of companies.
- First wi-fi protocol: WEP completely broken.
- Second wi-fi protocol: WPA used own cipher, broken
- Third protocol WPA2 uses AES in CCM mode.
  - Security is based on a password known to the access point and client.
  - Password can be bruteforced offline.
  - KRACK attack
- WPA3 new proposed protocol with ECDH.

Simplified WPA 2



Decrypting WPA with Wireshark

- nonce (in 4-way handshake), + password + SSID gives you the key

- If you tell wireshark the password and SSID it will decrypt WPA traffic.
  - More details here: https://wiki.wireshark.org/HowToDecrypt802.11
  - You should not be able to read others traffic!

- If you have Na, Nb, SSID and encrypted message you can brute force the password offline.
  - https://www.shellvoide.com/wifi/hashcat-guide-how-to-brute-force-crack-wpa-wpa2/

Protocol Goals

- Keys set up are **secret and fresh**.
- **Authenticate: both parties**, i.e., both are talking to who they really believe they are talking to.
- **Forward Secrecy.**
- **No weak cipher.**
- **Key Compromise Impersonation Attack.**
- **Protocol should not be complex or have lots of different modes.**

Protocol Goals

- Keys set up are **secret and fresh**.
- **Authenticate: both parties**, i.e., both are talking to who they really believe they are talking to.
- **Forward Secrecy.**
- **No weak cipher.**
- **Key Compromise Impersonation Attack.**
- **Protocol should not be complex or have lots of different modes.**
- **No offline guessing attacks against weak secrets**

WPA3

- Adds the “Dragonfly” Protocol to set up a shared secret based on the password.
  - Observer than knows the password cannot learn key
  - Cannot brute force low entropy passwords.
  - Add forward secrecy.

- It then runs classic 4-way handshake.
  - 4-way handshake also set up group keys etc.

WPA3 Dragonfly Protocol

- N.B. P is private password (unlike ECDH)
- Cannot learn r or m from s and E.
- Alice calculates:

$r_A.(s_B.P + E_B)$

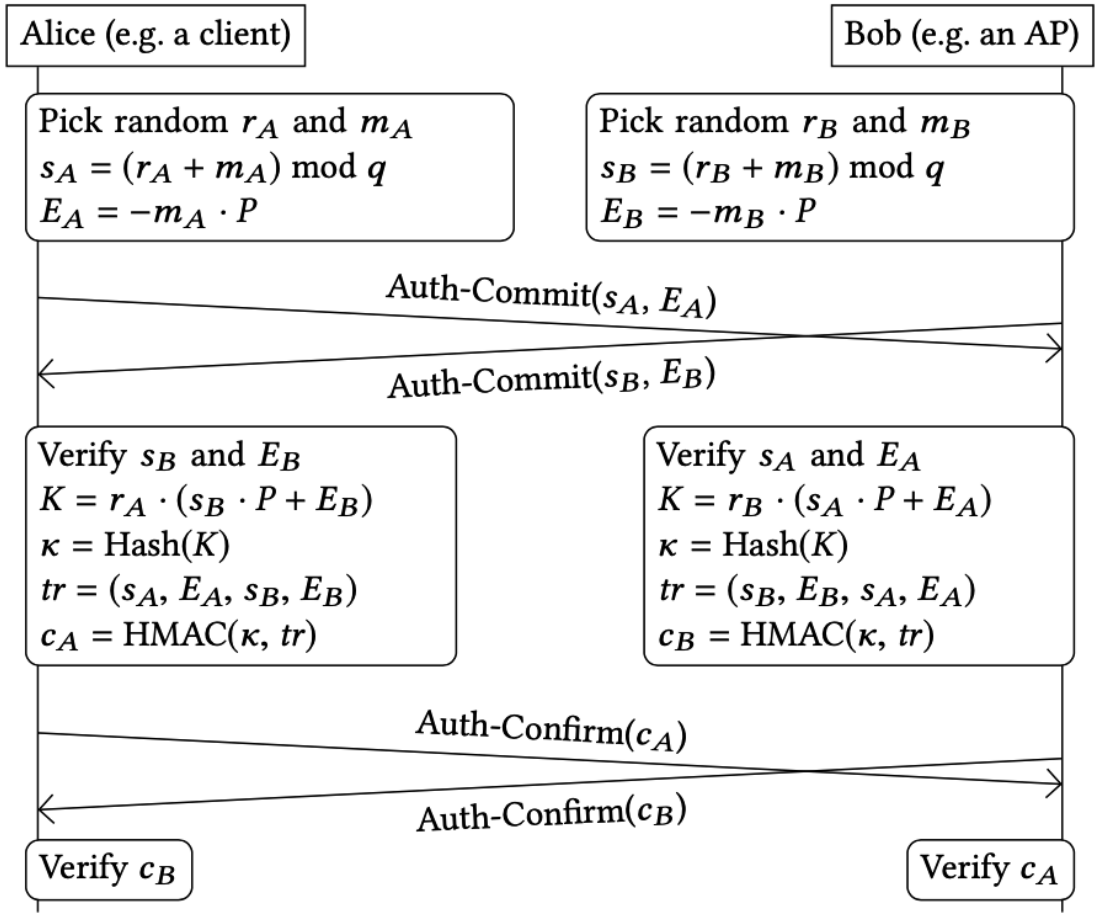
$= r_A.((\ r_B+m_B\ ).P + E_B)$

$= r_A.((\ r_B+m_B\ ).P - m_B.P)$

$= r_A.(\ r_B+m_B\ ).P - r_A.m_B.P)$

$= r_A.r_B.P + r_A.\ m_B.P - r_A.m_B.P$

$= r_A.r_B.P$



https://wpa3.mathyvanhoef.com

WPA3 Dragonfly Protocol

- N.B. P is private password (unlike ECDH)
- Cannot learn r or m from s and E.
- Bob calculates:

$r_A.(s_A.P + E_A)$

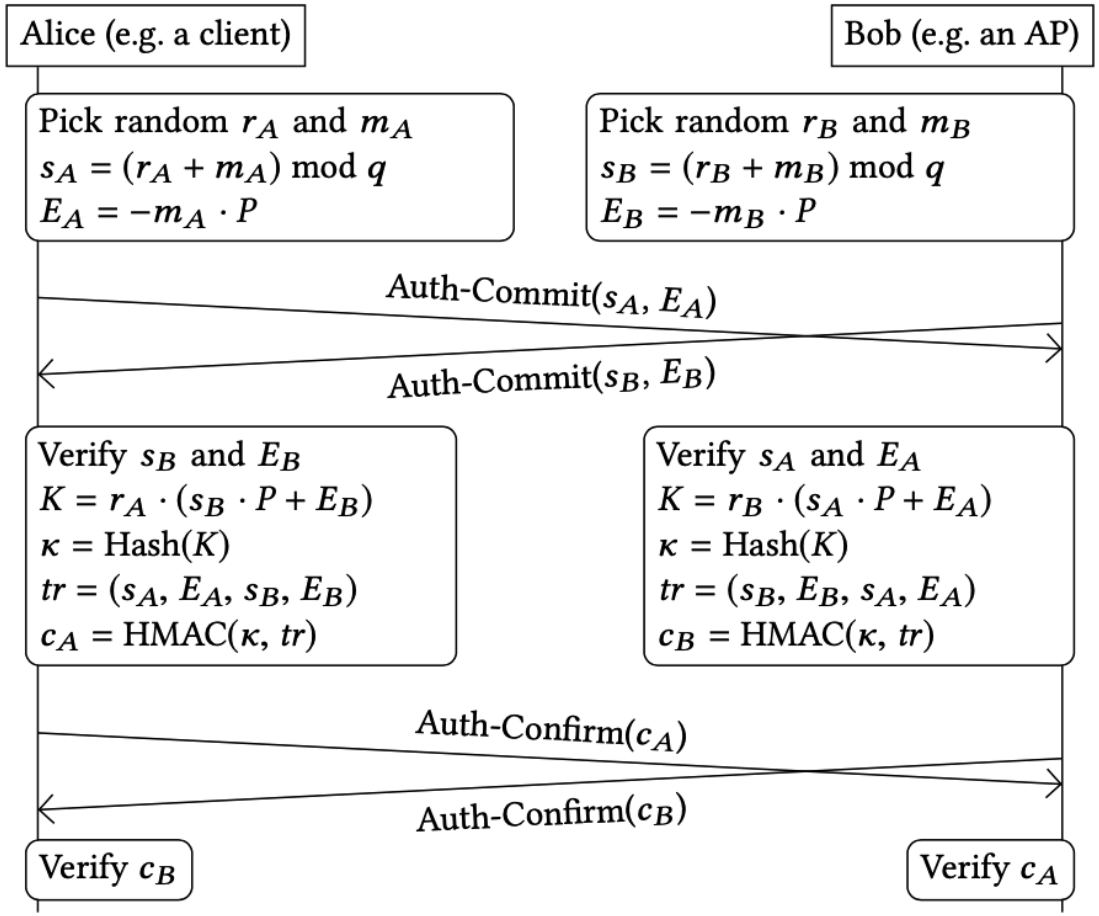
$= r_B.((\ r_A+m_A\ ).P + E_A)$

$= r_B.((\ r_A+m_A\ ).P - m_A.P)$

$= r_B.(\ r_A+m_A\ ).P - r_B.m_A.P)$

$= r_B.r_A.P + r_B.m_A.P - r_B.m_A.P$

$= r_B.r_A.P$



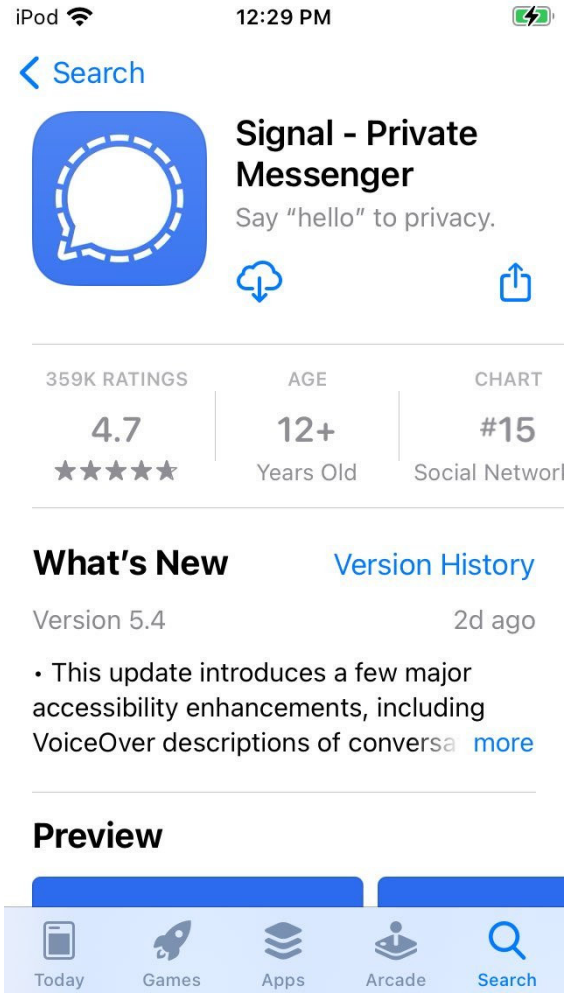
This and following figures from https://wpa3.mathyvanhoef.com

WPA3

- Adds the “Dragonfly” Protocol to set up a shared secret based on the password.
  - Observer than knows the password cannot learn key
  - Cannot brute force low entropy key.
  - Add forward secrecy.
- It then runs classic 4-way handshake.
  - 4-way handshake also set up group keys etc.

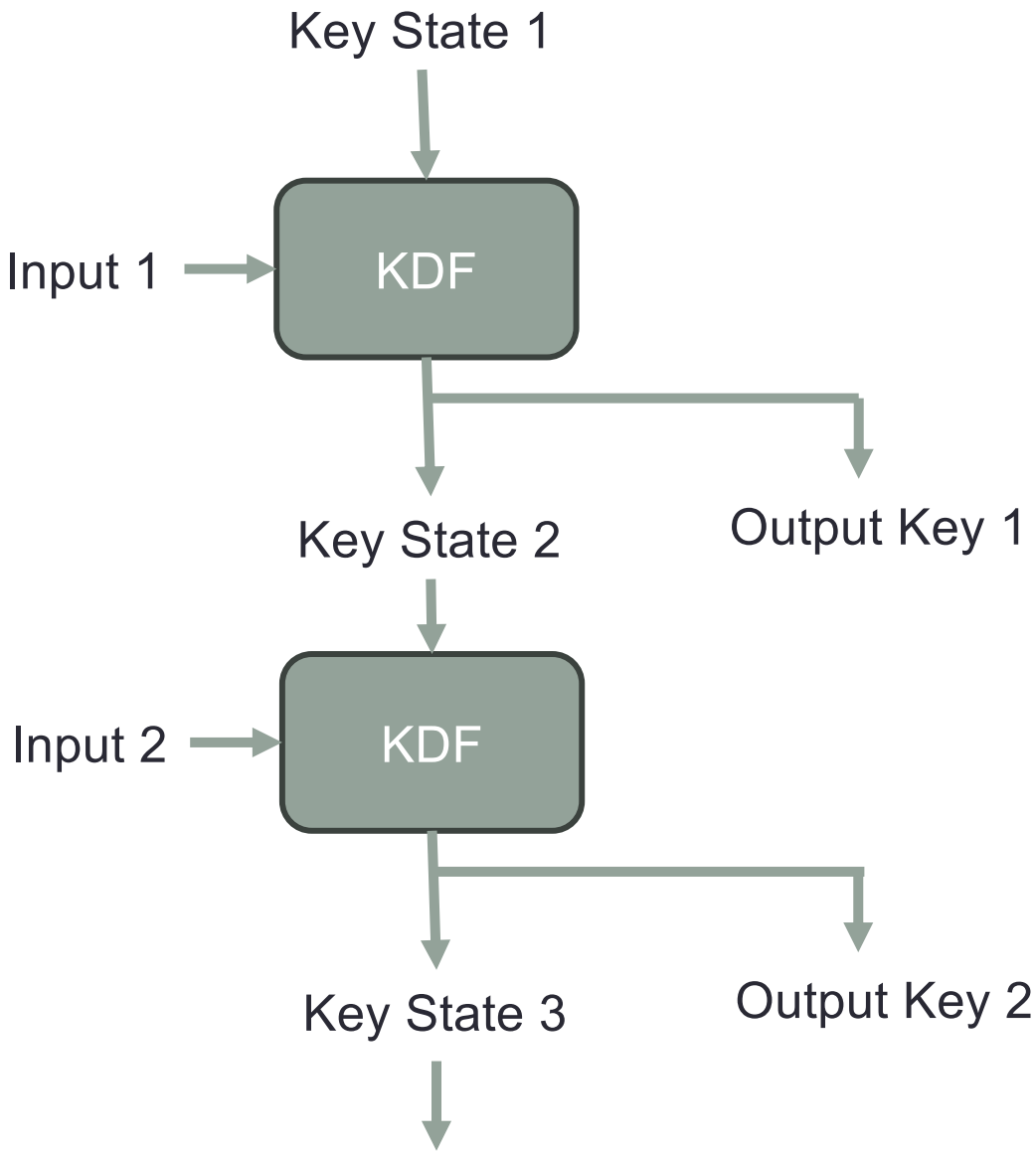
The Signal Protocol

- The most secure messaging app.
- Built to stop state actors.
- Uses lots of Elliptic Curve Diffie Hellman (ECDH).
- Uses Key Derivation Functions (KDF).
- Long lived sessions for chats.



Key Derivation Functions (KDF)

- KDF takes any (e.g. 1 or 2) input(s) and produces a key and a key state.
- Output Key used as session key in protocol.
- Key State used to generate more keys.
- One way function. (e.g., SHA256, with 1st 128 bits the state and 2nd 128 bits the key).



Messaging Protocol

- Signal runs an always online server that you can use to look up peoples keys, based on their phone number.
- Server MUST NOT be able to read peoples messages.
- The receiver may be offline, so protocol must be asynchronous.
- Everyone sends to the server:
  - A long-term public key g^{ltkA}
  - A medium-term, public key g^{mtkA} (signed with the long-term key).
  - A collection of signed keys g^{otk1A} ,g^{otk1A},g^{otk1A} , …

Establishing a Session

- Alice gets Bob's keys from the server: g^{ltkB}, g^{mtkB}, g^{otkB}
- Alice generates new "ephemeral"/fresh DH key ephA, g^{ephA}.
- Alice generates the key state:
  - State | key = KDF((g^{mtkB})^{ltkA} | (g^{ltkB}) ^{ephA} | (g^{mtkB})^{ep}

N.B. signal docs write g^{xy} as DH(x,gy)

  - $g^{mtkB.ltkA}$ A's unique ID bound to this session from Bob
  - $g^{ltkB.ephA}$ A's fresh key bound to Bob's ID
  - $g^{mtkB.ephA}$ A's fresh key bound to the session from Bob
  - $g^{otkB.ephA}$ A's fresh key bound one time key (stops replay)
- Alice sends to Bob: g^{ltkA}, g^{ephA} ,g^{otkB},$\{Message\}_{key}$

Signal Protocol

- A -> S: B
- S -> A: g^{ltkB}, g^{mtkB} g^{otkB}
- A ->B: g^{ltkA}, g^{ephA} ,g^{otkB}, $\{Message\}_{key}$

Message to Bob goes via the server, but the server can't read it.

Bob does not have to be online.

Bob then deletes his one-time key otkB

- Alice: State | key = $KDF((g^{mtkB})^{ltkA} \mid (g^{ltkB})^{ephA} \mid (g^{mtkB})^{ephA} \mid (g^{otkB})^{ephA})$
- Bob: State | key = $KDF((g^{ltkA})^{lmkB} \mid (g^{ephA})^{ltkBA} \mid (g^{ephA})^{mtkB} \mid (g^{ephA})^{otkB})$

Session Key Continually Updated.

- Sender generates x, g^x and sends g^x with messages:
- A ->B: $\{Message\}_{key1}$, g^x
- Bob generates y, g^y and updates state and key:

  NewState,key2 = KDF(OldState,g^{xy})
- Bob uses the new key: B ->A: $\{Message\}_{key2}$, g^y
- To send two messages in a row Alice does:

  NewState,key2 = KDF(OldState)

Signal Protocol

- A -> S: B
- S -> A: g^{ltkB}, g^{mtkB}, g^{otkB}
- A ->B: g^{ltkA}, g^{ephA} ,g^{otkB}, $\{Message\}_{key1}$, g^{x1}
- B -> A: $\{Message\}_{key2}$, g^{y1}
- A -> B: $\{Message\}_{key3}$, g^{x2}
- B -> A: $\{Message\}_{key4}$, g^{y2}
- A -> B: $\{Message\}_{key5}$, g^{x3}
- …
- N.B. This is simplified, the full protocol has trees of keys for multiple sessions and chats.

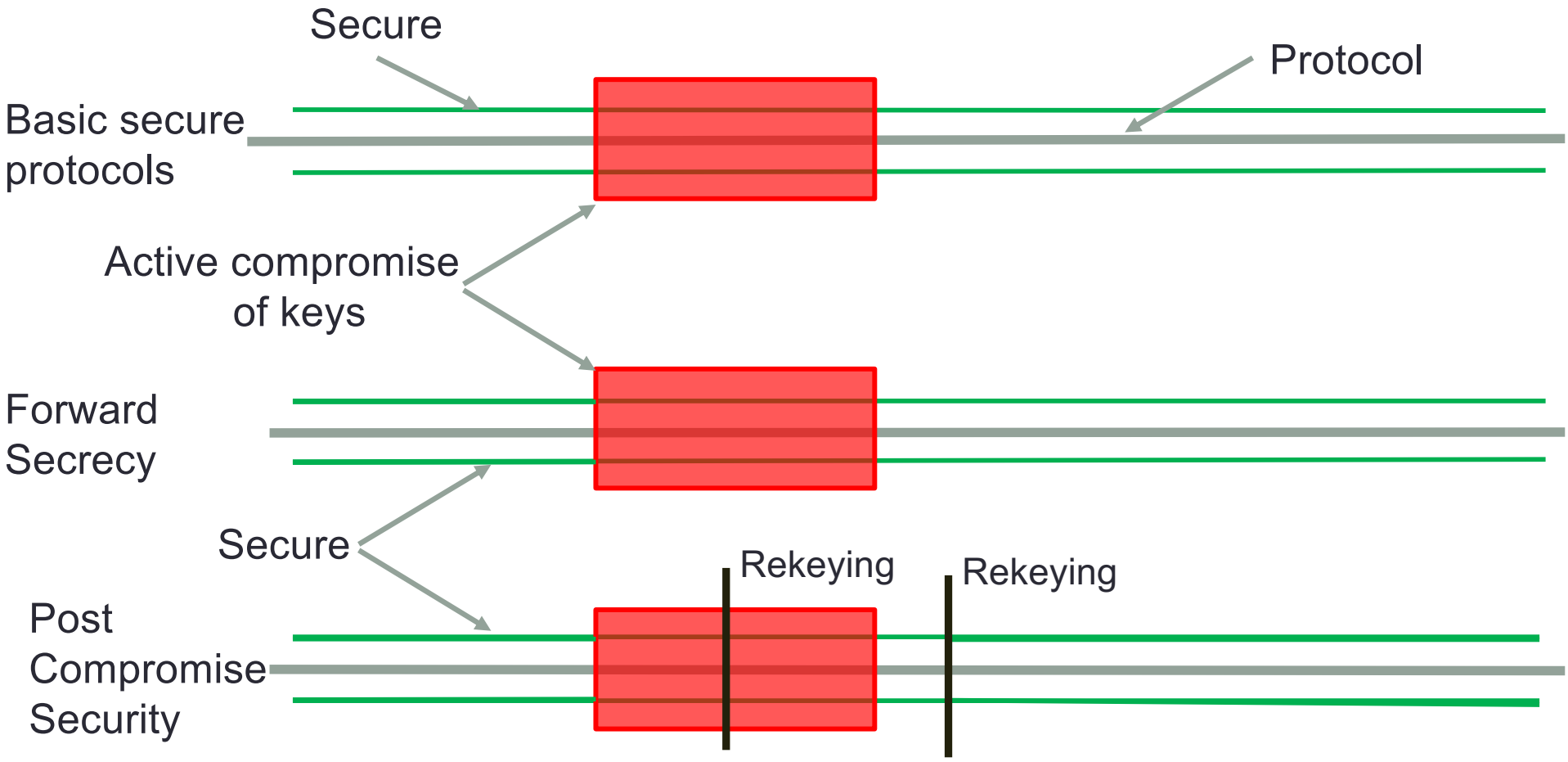
- State1 | key1
  =. KDF(($g^{mtkB.ltkA}$ | ($g^{ltkB.ephA}$ | ($g^{mtkB.ephA}$ | ($g^{otkB.ephA}$))
- State2 | key2
  = KDF(State1 | ($g^{y1.x1}$))
- State3 | key3
  = KDF(State2 | ($g^{y1.x2}$))
- State2 | key4
  = KDF(State1 | ($g^{y2.x2}$))
- State3 | key5
  = KDF(State2 | ($g^{y2.x3}$))

Post Compromise Security

- Very strong protection against key compromise.
- If attacker gets a long-term key, they cannot break existing sessions.
- If the attacker gets all keys, they will lose access after rekeying, unless they actively MITM the handshake.
- Also stops the attacker getting a large amount of traffic encrypted with the same key.
  - Stops side channel attacks
  - Or yet to be discovered crypto attacks

Post Compromise Security

- If attacker complete compromises a system, then stops post compromise security will restore the protection



Protocol Goals

- Keys set up are secret and fresh.
- Authenticate: both parties to each other
- Forward Secrecy.
- No weak cipher.
- Minimum possible damage from key leakage.
- As simple as possible.
- No offline guessing attacks against weak secrets
- Post compromise security.
- Reveals as little meta information as possible.

What you need to know.

- Understand the protocol notation, what Diffie Hellmen does and how it works.
- Understand the protocol goals, why (and when) they are needed, and how to get them.
- You don’t need to memorize protocols; I will remind you of them if I ask about them on the exam.
- But please make sure you understand everything in the slides.
- Lots of related reading on the Canvas page, but only content in the slides is needed.